
System Design — Volume I

Foundations & Core Patterns | Staff Engineer Level

Apple Staff / Principal Engineer Interview Preparation

System Design Series — Printer-Friendly Edition

System Design — Volume I

Apple Inc Staff Engineer Interview Prep | Foundations & Core Patterns

■ = Frequently asked at Apple | ■ = Basic | ■ = Intermediate | ■ = Advanced

>

Volume I covers Staff Engineer-level topics: scalability fundamentals, core building blocks, and classic design problems. **Volume II** covers Senior Staff Engineer-level topics: distributed systems theory, advanced consistency patterns, and complex multi-system designs.

Table of Contents

1. [System Design Fundamentals](#chapter-1-system-design-fundamentals)
 2. [Load Balancing](#chapter-2-load-balancing)
 3. [Caching Strategies](#chapter-3-caching-strategies)
 4. [Database Scaling — Replication & Sharding](#chapter-4-database-scaling)
 5. [SQL vs NoSQL — Choosing the Right Database](#chapter-5-sql-vs-nosql)
 6. [API Design — REST, GraphQL, gRPC](#chapter-6-api-design)
 7. [Message Queues & Event-Driven Architecture](#chapter-7-message-queues--event-driven-architecture)
 8. [Microservices Architecture](#chapter-8-microservices-architecture)
 9. [Classic Design Problem: URL Shortener](#chapter-9-design-url-shortener)
 10. [Classic Design Problem: Twitter / News Feed](#chapter-10-design-twitter--news-feed)
 11. [Classic Design Problem: Notification System](#chapter-11-design-notification-system)
 12. [Classic Design Problem: File Storage (Dropbox)](#chapter-12-design-file-storage-dropbox)
-

Chapter 1: System Design Fundamentals

Q1 ■ ■ What are the core properties of a well-designed distributed system?

The Four Pillars

■	SCALABILITY	■	Handle growing load without redesigning	■
■	AVAILABILITY	■	Uptime – system responds when needed	■
■	RELIABILITY	■	Correctness – correct results even on failure	■
■	PERFORMANCE	■	Latency (single request) + Throughput (total)	■

Latency vs Throughput

Latency = time for ONE request to complete (P50, P95, P99)

Throughput = requests served per second (RPS / QPS)

Analogy: A highway

Latency = time for one car to travel from A to B

Throughput = cars passing a checkpoint per hour

You can have:

- High throughput, high latency (many slow requests batched)
- High throughput, low latency (ideal – hard to achieve)
- Low throughput, low latency (fast but few requests handled)

Real numbers to know:

L1 cache access: 0.5 ns

L2 cache access: 7 ns

RAM access: 100 ns

SSD random read: 150 μ s

Network (same DC): 0.5 ms

Network (cross-DC): 10-100 ms

HDD seek: 10 ms

Packet: CA → Europe: 150 ms

Availability — The Nines

Availability	Downtime/year	Downtime/month	Downtime/day	
99%	3.65 days	7.2 hours	14.4 minutes	
99.9%	8.7 hours	43.8 minutes	1.44 minutes	
99.99%	52 minutes	4.4 minutes	8.6 seconds	
99.999%	5.3 minutes	26 seconds	0.86 seconds	← "five nines"
99.9999%	31 seconds	2.6 seconds	86 ms	← Apple Pay target

Key Interview Framework — How to answer any design question

- Step 1: CLARIFY requirements (5 minutes)
- Functional: what must the system DO?
 - Non-functional: scale, latency, availability, consistency needs
 - Constraints: read vs write ratio, data size, geographic scope
- Step 2: ESTIMATE scale (5 minutes)
- Daily active users (DAU), requests per second (RPS)
 - Storage needs (data size × retention)
 - Bandwidth (request size × RPS)
- Step 3: HIGH-LEVEL design (10 minutes)
- Draw the major components: clients, LB, services, DB, cache
 - Data flow: where does data come in, where does it go?
- Step 4: DEEP DIVE into critical components (15 minutes)
- Database schema and choice
 - Bottlenecks and how you'd solve them
 - Trade-offs you're making
- Step 5: WRAP UP (5 minutes)
- Failure scenarios (what if DB goes down?)
 - Monitoring / alerting
 - What you'd do with more time

Q2 ■ ■ What is horizontal vs vertical scaling?

Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
	
Bigger machine	More machines
[Server: 4 cores, 16GB RAM] ↓ upgrade [Server: 64 cores, 256GB RAM]	[Server][Server][Server][Server] ↑ ↑ ↑ ↑ same small machines, load balanced
Pros: Simple, no app changes	Pros: Unlimited scale, fault tolerant
Cons: Hard limits, single point of failure, expensive	Cons: App must be stateless, complexity of coordination
Rule of thumb:	
Start vertical (simpler)	
Switch to horizontal when vertical limit approached or HA needed	

Q3 ■ ■ What are SLO, SLA, and SLI? How do error budgets work?

SLI (Service Level Indicator) – the MEASUREMENT
 "Our P99 latency this week was 120ms"
 "Our availability this month was 99.95%"

SLO (Service Level Objective) – the TARGET (internal promise)
 "P99 latency must be < 200ms"
 "Availability must be ≥ 99.9%"

SLA (Service Level Agreement) – the CONTRACT (external, legal)
 "If availability < 99.9% in a month, customer gets 10% refund"

Error Budget = 100% - SLO target
 99.9% SLO → error budget = 0.1% = 43.8 minutes/month
 If you use up your budget, freeze new deployments until next month
 If budget remains, you can safely ship risky changes

Why Apple cares: Error budgets make reliability vs velocity trade-offs objective. Engineers can't ship if reliability is already below target.

Chapter 2: Load Balancing

Q4 ■ ■ What is a load balancer? What algorithms does it use?

Plain English First

A load balancer is the **traffic cop** in front of your servers. Instead of all traffic hitting one server (which would be overwhelmed), the LB spreads requests across many servers. When a server dies, the LB stops sending traffic to it.

```
Without LB:
  Client → Server1 (overloaded, crashes)

With LB:
  Client → Load Balancer → Server1 (30% load)
                        → Server2 (35% load)
                        → Server3 (35% load)
```

Load Balancing Algorithms

1. Round Robin (default)
Req 1 → Server1, Req 2 → Server2, Req 3 → Server3, Req 4 → Server1...
Best for: identical, stateless servers with similar request costs
Problem: ignores server load – a slow server still gets requests
2. Weighted Round Robin
Server1 (weight=3) → Server2 (weight=1)
3 out of 4 requests go to Server1 (more powerful machine)
Best for: heterogeneous hardware
3. Least Connections
Always send to server with fewest active connections
Best for: varying request duration (some requests take 1ms, others 5s)
4. Least Response Time
Send to server with lowest avg response time + fewest connections
Best for: performance-sensitive APIs
5. IP Hash / Sticky Sessions
hash(client_ip) % server_count → always same server
Best for: stateful apps (sessions, websockets)
Risk: if one IP generates huge traffic, one server is overloaded
6. Random with Two Choices (Power of Two)
Pick 2 servers randomly, send to the less loaded one
Best for: large clusters – near-optimal with much less overhead than pure least-connections (no global state needed)

L4 vs L7 Load Balancers

- L4 (Transport Layer – TCP/UDP)**
Routes based on IP + port only
Cannot see HTTP headers, URLs, cookies
Fast – minimal processing
Example: AWS NLB, HAProxy in TCP mode
Use for: raw throughput, non-HTTP protocols (database, gaming)
- L7 (Application Layer – HTTP/HTTPS)**
Routes based on URL path, headers, cookies, body
Can do: SSL termination, content-based routing, A/B testing, auth
Slightly more CPU overhead
Example: AWS ALB, Nginx, Envoy
Use for: HTTP APIs, microservices, gRPC
- Examples of L7 routing:
/api/search/* → Search service cluster
/api/payments/* → Payment service cluster (isolated, PCI compliant)
/static/* → CDN or static file server
Header: X-Beta=true → Canary cluster (5% of users)

Health Checks

Active health check: LB pings each server every N seconds
GET /health → 200 OK → keep sending traffic
GET /health → 500 / timeout → mark server DOWN, stop sending traffic

Passive health check: LB watches real traffic
If 5 consecutive requests to Server2 get 5xx → mark as unhealthy

Spring Boot – expose health endpoint:
GET /actuator/health → {"status":"UP"}
LB checks this every 5s
If 2 consecutive checks fail → server removed from rotation

Q5 ■ What is a reverse proxy? How is it different from a load balancer?

Forward Proxy: client uses it to reach the internet (corporate proxy)
Client → Forward Proxy → Internet

Reverse Proxy: sits in front of YOUR servers (the internet uses it to reach you)
Internet → Reverse Proxy → Your Server

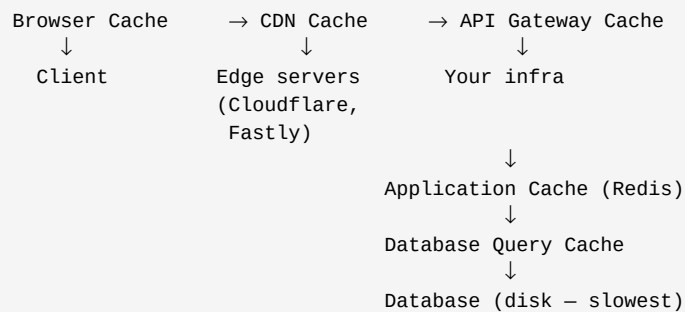
Reverse proxy can do:

- ✓ Load balancing (route to multiple backends)
- ✓ SSL termination (decrypt HTTPS, talk HTTP internally)
- ✓ Compression (GZIP responses)
- ✓ Static file serving (serve /static/* itself, don't bother backend)
- ✓ Caching (cache responses, reduce backend load)
- ✓ Rate limiting (block abusive IPs)
- ✓ Authentication (check JWT before request reaches your service)
- ✓ DDoS protection (absorb traffic spikes)

Common: Nginx, Envoy, HAProxy, AWS ALB, Cloudflare

Chapter 3: Caching Strategies

Q6 ■ ■ Explain the different caching layers in a web system.



Where to cache what

Layer	What to cache	TTL
Browser	Static assets (JS, CSS, img)	Days-months (with versioned URLs)
CDN	Static assets + API responses	Minutes-hours
App cache (Redis)	Session data, computed results	Seconds-minutes
DB query cache	Expensive aggregations	Minutes-hours
Read replicas	Read-heavy queries (no cache)	N/A – always fresh

Q7 ■ ■ What are cache eviction policies? Explain LRU, LFU, TTL.

TTL (Time To Live) – evict after a fixed time
key expires in 5 minutes regardless of access
Best for: data that goes stale (stock prices, exchange rates)
Problem: cold start after expiry – every client hits DB simultaneously (cache stampede)

LRU (Least Recently Used) – evict what was accessed longest ago
Cache is full → remove the entry not used for the longest time
Best for: temporal locality – recently used data tends to be used again
Used by: Redis (maxmemory-policy allkeys-lru)

LFU (Least Frequently Used) – evict what is accessed least often
Cache is full → remove the entry accessed fewest times
Best for: popularity-based data (viral content stays, niche content evicted)
Problem: newly added items start with low count – unfairly evicted

FIFO (First In, First Out) – evict oldest entry
Simple but ignores access patterns – rarely used in practice

Write-Around – write goes directly to DB, bypass cache
Cache only on read (lazy loading)
Best for: write-once-read-rarely data

Write-Through – write to cache AND DB simultaneously
Cache always consistent, but write latency increases
Best for: read-heavy data where staleness is unacceptable

Write-Behind (Write-Back) – write to cache, async write to DB later
Very fast writes, eventual DB consistency
Risk: cache crash before flush = data loss
Best for: high write throughput where slight data loss is tolerable (counters, analytics)

Cache Stampede (Thundering Herd) — and how to fix it

Problem:

10,000 users request /products/featured
Cache expires at 3:00:00 AM
All 10,000 requests see cache miss simultaneously
All 10,000 hit the database simultaneously → DB overloaded, crashes

Solutions:

1. Mutex / Lock – only one thread rebuilds the cache

```
if (cache.get("featured") == null) {  
    if (lock.tryLock(500ms)) {  
        // I got the lock – rebuild cache  
        data = db.fetchFeatured();  
        cache.set("featured", data, 60s);  
        lock.unlock();  
    } else {  
        // Someone else is rebuilding – wait briefly and retry  
        Thread.sleep(50ms);  
        return cache.get("featured"); // Should be warm now  
    }  
}
```

2. Probabilistic Early Recomputation

Before TTL expires, probabilistically start refreshing
if (ttl_remaining < random(0, max_delta)) { refresh(); }
Spreads refreshes over time – no thundering herd

3. Stale-While-Revalidate

Return stale data immediately
Trigger async background refresh
Next request gets fresh data
Used by: CDNs (Cache-Control: stale-while-revalidate=30)

Q8 ■ ■ What are cache invalidation strategies?

"There are only two hard things in Computer Science: cache invalidation and naming things." – Phil Karlton

Strategy 1: TTL-based (simple, most common)

Set an expiry – accept that data may be stale up to TTL seconds

Best for: data where slight staleness is acceptable (product catalog)

Strategy 2: Event-driven invalidation (complex, consistent)

When data changes in DB, immediately delete/update cache entry

Publisher: after DB update → publish "product:123:updated" event

Subscriber (cache service): delete cache["product:123"]

@Service

```
public class ProductService {  
    @CachePut(value = "products", key = "#product.id")  
    public Product update(Product product) {  
        return productRepository.save(product); // Cache updated atomically  
    }  
  
    @CacheEvict(value = "products", key = "#id")  
    public void delete(Long id) {  
        productRepository.deleteById(id); // Cache cleared on delete  
    }  
}
```

Strategy 3: Write-Through (always consistent)

Every write updates cache + DB together

No staleness – but write path is slower

Strategy 4: Cache-Aside (Lazy Loading – most flexible)

App checks cache → miss → reads DB → writes to cache → returns data

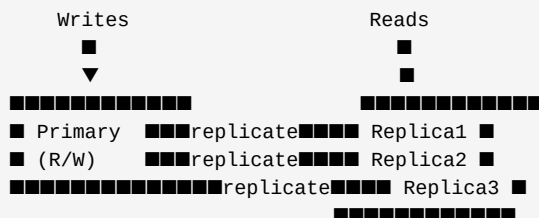
Simple, only caches what's actually used

Risk: cache miss on first request (cold start)

Chapter 4: Database Scaling

Q9 ■ ■ What is database replication? How does it help?

Primary-Replica (Master-Slave) Replication:



Benefits:

- Read scaling: distribute read queries across replicas
- HA: if primary fails, promote a replica to primary
- Reporting: run heavy analytics queries on replica, don't impact primary

Replication lag:

- Writes go to primary → async copy to replicas → replicas may be 10-500ms behind
- Problem: user writes a post, then immediately reads it → hits replica → doesn't see it!
- Solution: read your own writes from primary for 1s after a write

Sync vs Async replication:

- Async: primary commits, sends to replica later – fast writes, possible data loss on failover
- Sync: primary waits for replica ACK before committing – no data loss, slower writes
- Semi-sync: at least 1 replica must ACK – balance of both (MySQL semi-sync)

Q10 ■ ■ What is database sharding? What are the trade-offs?

Plain English First

Sharding = splitting your database into multiple independent pieces (**shards**), each handling a subset of data. Think of a phone book split alphabetically: A–G on server 1, H–P on server 2, Q–Z on server 3.

Without sharding:

Single DB → 100M rows → slow, single point of failure

With sharding ($\text{user_id} \% 3$):

Shard0: user_ids 0, 3, 6, 9... (on DB server 0)

Shard1: user_ids 1, 4, 7, 10... (on DB server 1)

Shard2: user_ids 2, 5, 8, 11... (on DB server 2)

Query for user 7 → $\text{shard} = 7 \% 3 = 1$ → go to DB server 1

Sharding Strategies

1. Range-Based Sharding

user_id 1-1M → Shard A

user_id 1M-2M → Shard B

user_id 2M-3M → Shard C

Pros: simple, range queries efficient

Cons: hot spots – most new users on latest shard (uneven load)

2. Hash-Based Sharding

$\text{shard} = \text{hash}(\text{user_id}) \% \text{num_shards}$

Pros: even distribution

Cons: range queries require ALL shards, resharding is hard

3. Directory-Based Sharding

Lookup table: user_id → shard_id (stored in Redis/DB)

Pros: flexible (move any user to any shard)

Cons: lookup table is a bottleneck and single point of failure

4. Geographic Sharding

US users → US shard (AWS us-east)

EU users → EU shard (AWS eu-west)

APAC users → APAC shard

Pros: data locality (low latency), data residency (GDPR)

Cons: cross-region queries are slow

Resharding Problem (Hot Shard)

Problem: One shard gets too much traffic (celebrity user, viral event)

$\text{hash}(\text{"taylorswift"}) \% 4 = \text{Shard2}$ → all Swift fans hit Shard2

Solutions:

1. Consistent hashing: adding shards only moves $\sim 1/n$ of keys (not everything)
2. Sub-sharding: split the hot shard into smaller ones
3. Account-level shard assignment: celebrities get dedicated shards
4. Application-level fan-out cache: pre-warm cache for celebrity data

Consistent Hashing (Solves Resharding)

Traditional hash: $\text{shard} = \text{key} \% N$

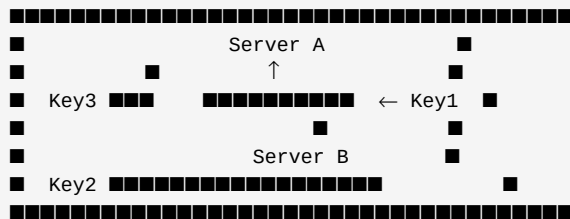
Add one shard ($N \rightarrow N+1$): almost ALL keys move to different shards

Cache invalidated, DB hammered $\rightarrow$ catastrophic

Consistent hashing:

Place N servers on a circular ring at hash positions

For each key: go clockwise on ring until you hit a server



Adding Server D: only keys between D's predecessor and D move $\rightarrow \sim 1/N$ keys move

Removing Server A: only A's keys move to A's successor $\rightarrow \sim 1/N$ keys move

Virtual nodes: each server gets K positions on the ring $\rightarrow$ more even distribution

Chapter 5: SQL vs NoSQL

Q11 ■ ■ When do you choose SQL vs NoSQL? Compare the major NoSQL types.

Decision Framework

Use SQL (PostgreSQL, MySQL) when:

- ✓ Data has clear relationships (foreign keys, joins needed)
- ✓ ACID transactions required (payments, inventory, orders)
- ✓ Schema is well-defined and stable
- ✓ Complex querying needed (GROUP BY, aggregations, JOINS)
- ✓ Data integrity is paramount

Examples: banking, e-commerce orders, user accounts

Use NoSQL when:

- ✓ Schema is flexible or evolving rapidly
- ✓ Massive scale (TB-PB of data, millions of writes/sec)
- ✓ Simple access patterns (key lookup, range scans)
- ✓ Horizontal scaling is a hard requirement
- ✓ Eventual consistency is acceptable

Examples: social media posts, IoT sensor data, shopping carts, logs

NoSQL Types

Type	Examples	Best For	
Key-Value	Redis, DynamoDB, Memcached	Sessions, caching, user preferences	0(1) reads/writes
Document	MongoDB, CouchDB, Firestore	Catalogs, user profiles, content management	Nested JSON, flexible schema
Wide-Column	Cassandra, HBase, Bigtable	Time-series, IoT, activity logs, analytics	Massive write throughput
Graph	Neo4j, Amazon Neptune, JanusGraph	Social networks, fraud detection, recommendations	Relationship traversal
Time-Series	InfluxDB, TimescaleDB, Prometheus	Metrics, monitoring, IoT	Optimized for time-ordered append + range queries

Cassandra (Wide-Column) — Deep Dive

Design around your queries (not your entities):

Data model for "get all tweets for a user, sorted by time":

```
CREATE TABLE tweets_by_user (
  user_id    UUID,
  posted_at  TIMESTAMP,
  tweet_id   UUID,
  content    TEXT,
  PRIMARY KEY (user_id, posted_at) ← user_id is partition key, posted_at is clustering key
) WITH CLUSTERING ORDER BY (posted_at DESC);
```

Query: `SELECT * FROM tweets_by_user WHERE user_id = ? LIMIT 20;`
 → Single partition read – extremely fast (no joins, no scatter-gather)

Rules:

- ✓ Denormalize – duplicate data to avoid joins
- ✓ One table per query pattern
- ✓ Partition key determines which node holds the data
- ✗ No joins, no transactions across partitions
- ✗ No ad-hoc queries

Chapter 6: API Design

Q12 ■ ■ What are REST best practices? What makes a good API?

REST Principles:

1. Resource-based URLs (nouns, not verbs)
 - ✓ GET /users/123/orders
 - ✗ GET /getUserOrders?userId=123
2. Correct HTTP methods
 - GET → read (safe, idempotent)
 - POST → create (not idempotent)
 - PUT → full replace (idempotent)
 - PATCH → partial update (idempotent)
 - DELETE → remove (idempotent)
3. Meaningful HTTP status codes
 - 200 OK → success
 - 201 Created → POST succeeded, resource created
 - 204 No Content → success, no body (DELETE, PUT)
 - 400 Bad Request → client error (validation failed)
 - 401 Unauthorized → not authenticated
 - 403 Forbidden → authenticated but not authorized
 - 404 Not Found → resource doesn't exist
 - 409 Conflict → duplicate, version mismatch
 - 429 Too Many Requests → rate limited
 - 500 Server Error → bug in your code
 - 503 Unavailable → overloaded, dependency down
4. Versioning strategy
 - URL versioning: /api/v1/users (most common – visible, cacheable)
 - Header versioning: Accept: application/vnd.api+json;version=2
 - Query param: /users?version=2 (avoid – breaks caching)
5. Pagination (never return unbounded lists)
 - GET /orders?page=0&size=20&sort=createdAt,desc
6. Consistent error response format
 - ```
{
 "status": 400,
 "error": "VALIDATION_FAILED",
 "message": "Email is required",
 "timestamp": "2026-05-07T10:00:00Z",
 "path": "/api/users",
 "traceId": "abc123" ← for debugging in logs
}
```

## Idempotency — Critical for Apple Pay Scale

Idempotent = calling the same request multiple times has same effect as calling once

GET, PUT, DELETE are idempotent  
POST is NOT idempotent by default

Problem: network timeout on payment → did the charge go through? Retry?  
If not idempotent: retry = double charge

Solution: Idempotency keys

Client generates: idempotency-key: uuid-abc123  
Server stores: (idempotency-key → result) in Redis

First call: process payment, store result in Redis with key  
Retry call: key already in Redis → return stored result, skip processing

POST /payments  
Headers: Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000  
Body: { "amount": 99.99, "currency": "USD" }

→ If client retries (same key), server returns cached result, no double charge

---

## Q13 ■ REST vs GraphQL vs gRPC — When to use each?

---

## REST

Request/Response over HTTP, JSON bodies

When: public APIs, browser clients, simple CRUD

Pros: simple, widely understood, HTTP caching works

Cons: over-fetching (too many fields), under-fetching (need multiple requests),  
versioning is hard

## GraphQL

Client specifies exactly what fields it needs

When: client-driven APIs (mobile apps with variable bandwidth), BFF pattern

Pros: no over/under fetching, strong typing, single endpoint

Cons: complex server implementation, N+1 risk (need DataLoader),  
HTTP caching harder (all POSTs to /graphql)

Example:

```
query {
 user(id: "123") {
 name
 email
 orders(last: 5) { ← only last 5 orders
 id
 total
 }
 }
}
```

## gRPC

Binary protocol (Protobuf), HTTP/2, streaming support

When: internal service-to-service communication, high throughput, streaming

Pros: 5-10× smaller payloads than JSON, strongly typed contracts,  
streaming (server-push, client-push, bidirectional)

Cons: not browser-native, binary (harder to debug), requires protobuf tooling

Apple use case:

iOS App → REST (JSON) → API Gateway → gRPC → Internal microservices

External: REST for simplicity

Internal: gRPC for performance

---

## Q14 ■ ■ How do you design rate limiting?

---

Why rate limit:

- Protect services from abuse (DDoS, scrapers, runaway clients)
- Ensure fair usage (one tenant can't starve others)
- Cost control (expensive AI APIs)

Rate limiting algorithms:

1. Fixed Window Counter

```
10 req / 1 min window
window = floor(now / 60)
counter[user_id][window]++
if counter > 10: reject
```

Problem: burst at window boundary

User sends 10 at 0:59, 10 at 1:01 → 20 requests in 2 seconds

2. Sliding Window Log

```
Store timestamp of each request in sorted set
Count requests in last 60 seconds
if count > 10: reject
```

Accurate but memory-intensive (stores all timestamps)

3. Token Bucket (most common)

```
Bucket holds tokens (max capacity = burst limit)
Tokens refill at rate R/sec
Each request consumes 1 token
If no tokens: reject
```

Allows bursts (drain bucket), then smooth flow at rate R

Implementation: Redis INCR + EXPIRE or Lua script for atomicity

4. Leaky Bucket

```
Requests enter a queue (bucket)
Processed at constant rate (leak)
If queue full: reject
```

Smooths out bursts – output always at constant rate

Used for: bandwidth shaping, outbound rate limiting

Redis implementation (Token Bucket):

```
KEYS[1] = "rate:user:123"
local tokens = redis.call('GET', KEYS[1])
if tokens == false then tokens = MAX_TOKENS end
if tokens > 0 then
 redis.call('SET', KEYS[1], tokens-1, 'EX', WINDOW)
 return 1 -- allowed
else
 return 0 -- rejected
end
```

Where to rate limit:

API Gateway (first line of defense) – by IP, by API key  
Service level – by user, by tenant, by endpoint

### Q15 ■ ■ Why use a message queue? What problem does it solve?

With a queue: orders go into the queue at whatever rate they arrive. The payment processor reads at its own pace (1,000/sec). No data lost, no overload — just a backlog that clears over time.

```
Order Service ■■■ [Queue] ■■■ Payment Service
 async (can be slow, can restart, can scale independently)
 order always accepted even if payment is busy
```

| Feature         | Apache Kafka                                       | RabbitMQ                                             |
|-----------------|----------------------------------------------------|------------------------------------------------------|
| Model           | Log-based (consumers track their own offset)       | Queue-based (broker tracks which messages delivered) |
| Retention       | Days/weeks (replay allowed)                        | Deleted after consumption                            |
| Throughput      | Millions msg/sec                                   | Tens of thousands msg/sec                            |
| Ordering        | Within partition                                   | Within single queue                                  |
| Consumer groups | Multiple – each gets ALL messages (replay)         | Multiple – each gets SOME messages (competing)       |
| Routing         | Topic → partitions                                 | Exchange → binding → queue                           |
| Best for        | Event streaming, audit log, data pipelines, replay | Task queues, RPC, complex routing, low-latency jobs  |

## Page 22 of 41

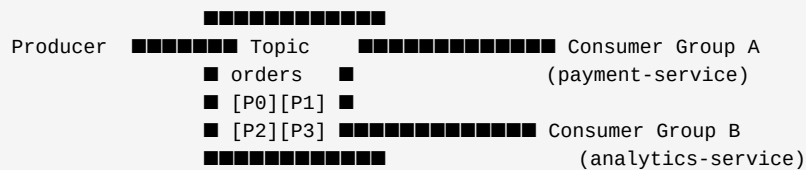
Topic: logical stream of events (like a DB table name)  
"orders", "payments", "user-events"

Partition: ordered, immutable log within a topic  
Topic "orders" might have 12 partitions  
Messages within a partition are strictly ordered  
Across partitions: no ordering guarantee

Offset: position in a partition (like an index)  
Consumer tracks its own offset – can re-read from any point

Consumer Group: set of consumers sharing the work  
Group "payment-service" has 4 consumers, 12 partitions  
→ each consumer handles 3 partitions ( $4 \times 3 = 12$ )  
→ parallel processing, ordered within each partition

Replication:  
Each partition has 1 leader + N-1 followers  
Writes go to leader, followers replicate  
If leader dies: follower is elected leader (no data loss with replication factor  $\geq 2$ )



## Q16 ■ ■ What is the Outbox Pattern? Why is it important?

### Plain English First

You have two operations that must both happen or both not happen:

1. Save order to DB
2. Publish "OrderPlaced" event to Kafka

Problem: What if you save to DB successfully, then Kafka is down? Event is never published — inconsistency.

The Outbox Pattern solves this with a simple trick: write the event to the DB (in the same transaction as the order), then a separate process reads from the DB and publishes to Kafka.

Step 1: Single DB Transaction

```
BEGIN;
 INSERT INTO orders (id, amount) VALUES (123, 99.99);
 INSERT INTO outbox (event_type, payload, published)
 VALUES ('OrderPlaced', '{"orderId":123}', false);
COMMIT;
→ Atomic: both succeed or both fail
```

Step 2: Outbox Relay (separate process – Debezium or custom)

```
SELECT * FROM outbox WHERE published = false;
→ Publish each event to Kafka
→ Mark as published = true
```

Result: DB and Kafka are always consistent

- If Kafka is down: events wait in outbox, published later
- If relay crashes: events stay unpublished, relay retries on restart
- No event is ever lost or published without the DB write succeeding

Debezium (CDC – Change Data Capture):

- Reads the PostgreSQL WAL (write-ahead log) stream
- Every INSERT into outbox table → Debezium streams it to Kafka
- Zero additional latency, no polling



## Chapter 8: Microservices Architecture

### Q17 ■ ■ What are microservices? When should you use them vs a monolith?

Monolith: one deployable unit containing all features

```

■ User Module ■ Order Module ■
■ Payment Mod ■ Inventory Mod ■
■ All in one JAR / one deploy ■

```

Microservices: many small, independent deployable services

[User Service] [Order Service] [Payment Service] [Inventory Service]  
Each has its own DB, deploys independently, communicates over network

When Monolith wins:

- ✓ Small team (< 10 engineers)
- ✓ Unclear domain boundaries (refactoring is cheap in a monolith)
- ✓ Low traffic (monolith is simpler to operate)
- ✓ Early stage (iterate fast, no deployment complexity)

Rule: "Start with a monolith, migrate when you feel the pain"

### When Microservices win:

- ✓ Multiple teams that need to deploy independently
- ✓ Different scaling needs (search scales differently than auth)
- ✓ Different tech stacks (ML model in Python, web in Java)
- ✓ Regulatory isolation (payment service in PCI-compliant environment)
- ✓ Fault isolation needed (payment bug shouldn't crash search)

## Service Decomposition Strategies

### By Business Capability (Domain-Driven Design):

|                  |                                         |
|------------------|-----------------------------------------|
| User Service     | → manages user accounts, authentication |
| Catalog Service  | → manages products, categories, search  |
| Order Service    | → manages order lifecycle               |
| Payment Service  | → manages payment methods, transactions |
| Notification Svc | → email, push notifications, SMS        |

### By Subdomain (Bounded Context):

- Each service owns its data (no shared DB)
- Services only communicate via APIs or events
- Loose coupling, independent deployability



```

// Resilience4j Circuit Breaker in Spring Boot
@Service
public class PaymentService {

 private final CircuitBreaker circuitBreaker;
 private final ExternalPaymentClient client;

 public PaymentResult charge(PaymentRequest request) {
 return CircuitBreaker.decorateSupplier(circuitBreaker,
 () -> client.charge(request)) // The actual call
 .get();
 }
}

// application.yml configuration
// resilience4j:
// circuitbreaker:
// instances:
// payment-service:
// slidingWindowSize: 10 # Track last 10 calls
// failureRateThreshold: 50 # Open if > 50% fail
// waitDurationInOpenState: 30s # Stay open for 30s
// permittedNumberOfCallsInHalfOpenState: 3

// Fallback – what to do when circuit is OPEN
@CircuitBreaker(name = "inventory-service", fallbackMethod = "fallbackInventory")
public InventoryStatus checkInventory(Long productId) {
 return inventoryClient.check(productId);
}

public InventoryStatus fallbackInventory(Long productId, Exception e) {
 // Return cached/default response instead of error
 return InventoryStatus.ASSUMED_AVAILABLE; // Optimistic fallback
}

```

---

## Q19 ■ What is service discovery? What is an API Gateway?

---

### Service Discovery:

Problem: microservices have dynamic IPs (containers restart, scale up/down)

Solution: a registry where services register themselves and discover each other

Client-side discovery (Eureka):

Service registers: POST /eureka/apps/PAYMENT-SERVICE {ip, port, health}

Client queries: GET /eureka/apps/PAYMENT-SERVICE → list of instances

## Client load balances among instances

Server-side discovery (AWS ALB, Kubernetes):

Client calls load balancer DNS name (stays fixed)

LB queries registry and routes to a healthy instance

Client doesn't need discovery logic

API Gateway:

Single entry point for all client requests

Responsibilities:

- ✓ Routing: /api/users/\* → User Service
- ✓ Auth: verify JWT before request reaches any service
- ✓ Rate limiting: 100 req/min per user (centralized)
- ✓ SSL termination: HTTPS → HTTP internally
- ✓ Request tracing: inject correlation ID into all requests
- ✓ Response caching: cache responses for GET endpoints
- ✓ Protocol translation: REST → gRPC for internal services

iOS App → [API Gateway] → [User Service]

→ [Product Service]

→ [Order Service]

Without gateway: each service implements auth, rate limiting, SSL separately

With gateway: handled once, DRY

# Chapter 9: Design — URL Shortener

---

## Q20 ■ ■ Design a URL shortening service like bit.ly

---

### Step 1: Clarify Requirements

#### Functional:

- Shorten a long URL to a short code (e.g., bit.ly/abc123)
- Redirect short URL to long URL
- Custom aliases (optional): bit.ly/my-custom-name
- Link expiry (optional)
- Analytics: click count, referrer, location

#### Non-functional:

- 100M new URLs per day (writes)
- 10B redirects per day (reads – 100:1 read/write ratio)
- Availability: 99.99% (redirects must always work)
- Redirect latency: < 10ms P99
- URL is available within 100ms of creation

### Step 2: Estimate Scale

Writes: 100M/day = ~1,160/sec

Reads: 10B/day = ~115,700/sec → very read-heavy

Storage per URL: 500 bytes (long URL + metadata)

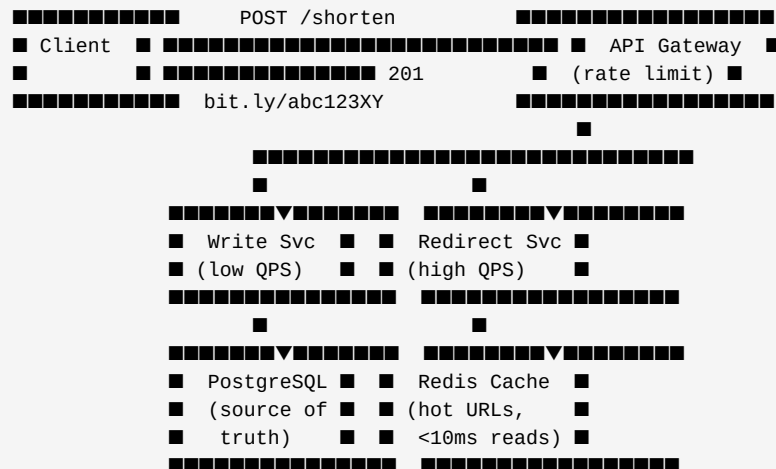
10 years retention:  $100M \times 365 \times 10 \times 500B = \sim 182 \text{ TB}$

Short code length:

Base62 (a-z, A-Z, 0-9) →  $62^6 = 56 \text{ billion unique codes}$

$62^7 = 3.5 \text{ trillion} \rightarrow 7\text{-character codes last for decades}$

### Step 3: High-Level Design



## Step 4: Short Code Generation

Option 1: Hash-based (MD5/SHA1 of long URL)  
 hash("https://apple.com/iphone15") → "a1b2c3d4e5f6..."  
 Take first 7 chars: "a1b2c3d"  
 Problem: collisions! Two URLs may produce same hash prefix

Option 2: Counter + Base62 encoding (recommended)  
 Maintain a global counter: 1, 2, 3, ...  
 Encode counter in Base62: 1→"1", 61→"z", 62→"10", 3844→"100"  
 Counter 1,000,000 → "4c92"

Global counter problem: single point of failure, bottleneck

Solution: Range-based counter allocation  
 Counter Service gives each Write Server a range (e.g., 1-10000)  
 Write Server 1 uses: 1, 2, ..., 10000  
 Write Server 2 uses: 10001, 20000  
 Each server works independently, no coordination needed

Option 3: Pre-generated codes (simplest for Apple scale)  
 Background job pre-generates 10M random 7-char codes, stores in "available" table  
 Write server: SELECT one code, mark as used → instantly available, no collision

```

CREATE TABLE available_codes (
 code VARCHAR(7) PRIMARY KEY,
 claimed_at TIMESTAMP
);
-- Pre-fill with 10M codes
-- Write service: SELECT code FROM available_codes WHERE claimed_at IS NULL LIMIT 1 FOR UPDATE SKIP LOCKED

```

## Step 5: Database Schema

```
CREATE TABLE short_urls (
 code VARCHAR(7) PRIMARY KEY,
 long_url TEXT NOT NULL,
 user_id BIGINT,
 created_at TIMESTAMP DEFAULT NOW(),
 expires_at TIMESTAMP,
 click_count BIGINT DEFAULT 0,
 is_active BOOLEAN DEFAULT TRUE
);

CREATE INDEX idx_short_urls_user ON short_urls(user_id);
CREATE INDEX idx_short_urls_expires ON short_urls(expires_at) WHERE expires_at IS NOT NULL;
```

## Step 6: Redirect Flow (Critical Path)

```
GET bit.ly/abc123XY
```

1. API Gateway → Redirect Service
2. Check Redis cache: GET "url:abc123XY"
  - HIT → return 301/302 with Location header → done in <5ms
  - MISS → query PostgreSQL, write to Redis with TTL, return redirect

Redis key: "url:abc123XY"  
 Value: "https://apple.com/iphone15"  
 TTL: 24 hours (popular URLs stay warm)

301 vs 302:

- 301 Permanent → browser caches, no server call next time → lower load, but can't track clicks
- 302 Temporary → browser always asks server → can count every click, update destination

Apple's choice: 302 for analytics, 301 for rarely-changed URLs

## Step 7: Analytics

Don't count clicks synchronously (would slow redirects)

Async approach:

- Redirect Service publishes event: {code, timestamp, referrer, user-agent, ip}
- Kafka topic "url-clicks"
- Analytics consumer reads, aggregates, writes to ClickHouse / BigQuery

Redis counter for real-time click count:

```
INCR "clicks:abc123XY" (atomic, O(1))
```

Sync to PostgreSQL every 5 minutes (batch update)

# Chapter 10: Design — Twitter / News Feed

---

## Q21 ■ ■ Design a Twitter-like news feed

---

### Step 1: Requirements

#### Functional:

- Users post tweets (280 chars)
- Users follow/unfollow other users
- Home timeline: see tweets from people you follow, reverse-chronological
- Notifications for mentions

#### Scale:

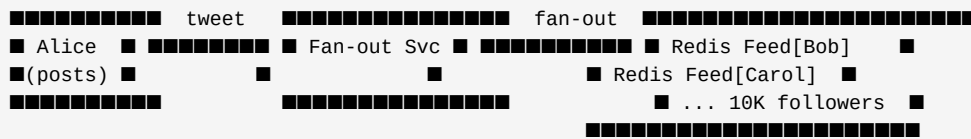
- 300M DAU
- 500M tweets per day → ~6,000 writes/sec
- Timeline reads: 10B/day → ~115,000 reads/sec
- Celebrity problem: some users have 100M followers (Elon Musk)

### Fan-Out Strategies



#### Fan-Out on Write (Push Model):

When Alice tweets → immediately write to all followers' feed lists (Redis Sorted Set)  
Read is O(1): just read your pre-computed feed list



Read: GET Redis Feed[Bob] → instant

Problem: celebrity with 10M followers → 10M Redis writes on each tweet → too slow!

#### Fan-Out on Read (Pull Model):

When Bob reads timeline → fetch tweets from all people Bob follows → merge → sort  
No pre-computation, always fresh

Read: 500 follows × DB query each → slow (500 queries or large JOIN)

Good for celebrities (no fan-out on write)

#### Hybrid (Twitter's actual approach):

Regular users (< 1M followers): fan-out on write → fast reads

Celebrities (≥ 1M followers): no fan-out → on read, merge celebrity tweets

Read timeline for Bob:

1. Fetch Bob's pre-computed feed from Redis (fast)
2. Find celebrities Bob follows
3. Fetch their recent tweets from a hot cache
4. Merge + sort + deduplicate → return top 200 tweets

## Data Model

```
-- Core tables (PostgreSQL)
CREATE TABLE users (
 user_id BIGINT PRIMARY KEY,
 username VARCHAR(50) UNIQUE,
 bio TEXT,
 follower_count INT DEFAULT 0,
 following_count INT DEFAULT 0
);

CREATE TABLE tweets (
 tweet_id BIGINT PRIMARY KEY, -- Snowflake ID (timestamp + machine + sequence)
 user_id BIGINT NOT NULL,
 content VARCHAR(280),
 media_urls TEXT[],
 created_at TIMESTAMP DEFAULT NOW(),
 like_count INT DEFAULT 0,
 retweet_count INT DEFAULT 0
);

CREATE TABLE follows (
 follower_id BIGINT,
 followee_id BIGINT,
 created_at TIMESTAMP DEFAULT NOW(),
 PRIMARY KEY (follower_id, followee_id)
);

-- Redis: home timeline cache
-- Key: feed:{user_id}
-- Value: Sorted Set of tweet_ids, score = timestamp (for ordering)
-- ZADD feed:bob_id timestamp tweet_id
-- ZREVRANGE feed:bob_id 0 49 → latest 50 tweet_ids
```

## Snowflake ID — Why not UUID or auto-increment?

UUID: random, 128-bit – causes index fragmentation, not sortable by time  
 Auto-increment: single point of failure, reveals count to competitors

Snowflake (Twitter's design):

64 bits = 1 sign + 41 timestamp + 10 machine\_id + 12 sequence  
 41-bit timestamp: milliseconds since epoch → works for 69 years  
 10-bit machine: 1024 different generators  
 12-bit sequence: 4096 IDs per millisecond per machine

=  $4096 \times 1024 \times 1000 = \sim 4$  billion IDs/second, globally unique, sortable!  
 tweet\_id 1746000000000000001 → you know it was created in May 2026

# Chapter 11: Design — Notification System

---

## Q22 ■ ■ Design a notification system (push, email, SMS)

---

### Requirements

#### Functional:

- Send push notifications (iOS APNs, Android FCM)
- Send emails (transactional + marketing)
- Send SMS
- Support templates with variable substitution
- Scheduled notifications (send at 9am user's local time)
- Priority levels (critical: payment failed; low: weekly digest)
- User preferences (opt-out, channel preferences)
- Delivery tracking (sent, delivered, opened, failed)

#### Scale:

- 10M push notifications / hour
- 1M emails / hour
- 100K SMS / hour

### Architecture

#### Event Sources:

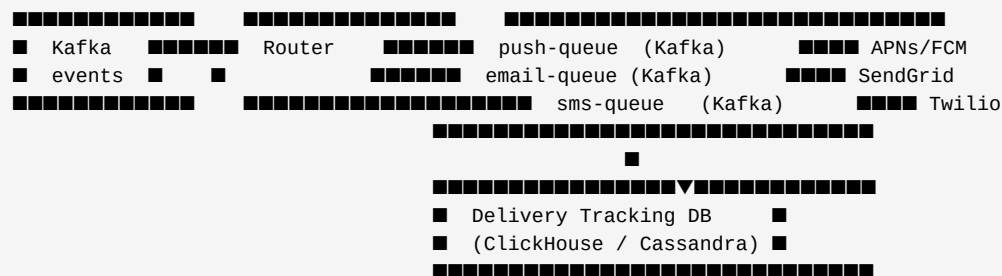
Order Service ■■  
Payment Svc ■■■■ Kafka ■■■■ Notification Router ■■■■ Channel Workers  
Marketing Tool ■■  
Scheduler ■■

#### Notification Router:

1. Fetch user preferences (does user want this type? which channels?)
2. Fetch user devices (for push: list of device tokens)
3. Apply template (fill in {name}, {amount}, {orderId})
4. Route to appropriate queue: push-queue / email-queue / sms-queue

#### Channel Workers (independent, auto-scaled):

Push Worker → APNs (iOS) / FCM (Android)  
Email Worker → SendGrid / SES  
SMS Worker → Twilio / AWS SNS



## Reliability — Retry & Dead Letter Queue

#### Transient failure (APNs timeout): retry with exponential backoff

- Attempt 1: immediately
- Attempt 2: +1 second
- Attempt 3: +2 seconds
- Attempt 4: +4 seconds
- Attempt 5: +8 seconds → if still fails → dead letter queue

#### Dead Letter Queue (DLQ):

- Failed notifications after max retries → DLQ topic
- On-call engineer investigates, can replay
- Alert if DLQ depth > threshold

#### Invalid device token (APNs feedback):

- User uninstalled app → APNs returns "invalid token"
- Remove token from DB, mark device as inactive (no more pushes to this device)

## Rate Limiting for Notifications

Per-user rate limits (avoid spamming):

- Max 5 push per hour per user (low priority)

- Max 1 push per 5 min per user (medium priority)

- No limit for critical (payment failed, security alert)

Per-tenant rate limits (SaaS):

- Marketing tier: 100K emails/hour

- Enterprise tier: 1M emails/hour

Implementation: Token bucket per user in Redis

- "notification\_bucket:{user\_id}:{priority}" → tokens

# Chapter 12: Design — File Storage (Dropbox)

---

## Q23 ■ ■ Design a file storage system like Dropbox

---

### Requirements

#### Functional:

- Upload, download, delete files
- File versioning (previous versions retrievable)
- Sync across devices (desktop, mobile, web)
- Share files/folders with others
- Conflict resolution (offline edits on multiple devices)

#### Scale:

- 500M users, 100M DAU
- 1 billion files stored
- 10 PB of total storage
- 1M uploads / day, 10M downloads / day
- File sizes: 1KB to 50GB

### Key Design Decisions

### 1. Chunking – break large files into chunks (4MB each)

Why:

- Resume interrupted uploads (only re-upload failed chunk)
- Parallel uploads (upload chunks concurrently)
- Delta sync (only transfer changed chunks, not entire file)
- Deduplication (same chunk shared across many users – huge storage savings)

Chunk deduplication (content-addressable storage):

- chunk\_hash = SHA256(chunk\_data)
- Check: does chunk\_hash already exist in S3?
- YES → just reference it, skip upload (zero bandwidth used!)
- NO → upload chunk, store at key = chunk\_hash

Real numbers: ~30% of chunks are duplicates → 30% storage saved

### 2. Metadata storage (PostgreSQL):

- Files table: file\_id, owner\_id, name, size, created\_at, current\_version\_id
- Versions table: version\_id, file\_id, chunk\_hashes[], created\_at
- Chunks table: chunk\_hash (PK), size, s3\_key

### 3. Blob storage: AWS S3 / Google GCS

- Each chunk stored at key = SHA256(chunk\_data)
- Files are reconstructed by assembling their chunks in order

## Upload Flow

Client uploads file "report.pdf" (100MB = 25 chunks of 4MB):

- Step 1: Client splits file into 25 chunks locally
- Step 2: For each chunk, client computes SHA256
- Step 3: Client asks metadata service: "which of these hashes do you have?"
- Step 4: Server returns missing chunk hashes (e.g., 18 of 25 are new)
- Step 5: Client uploads only 18 new chunks to S3 via pre-signed URLs
- Step 6: Client sends file manifest to metadata service:  
{name: "report.pdf", chunks: [hash1, hash2, ..., hash25]}
- Step 7: Metadata service creates Version record linking to all chunks

Result: Only 7 chunks (28MB instead of 100MB) transferred if 7 chunks already existed

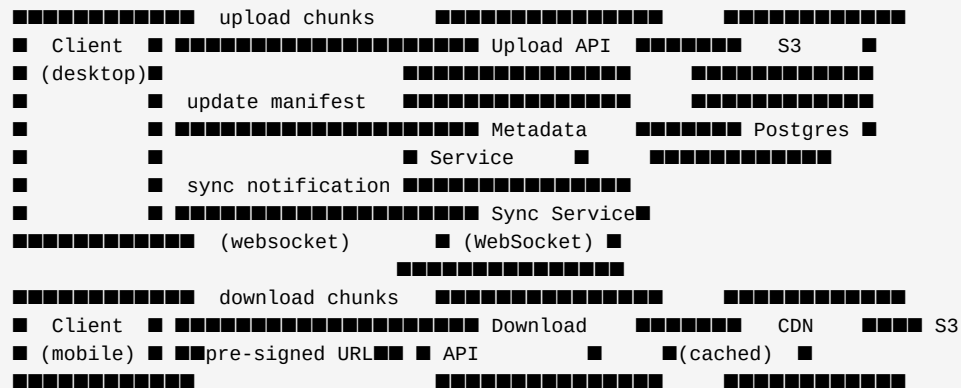
## Sync Flow

Device A makes change → local metadata updated → change event sent to sync service  
Sync service → notify all other devices via WebSocket / long poll → devices pull delta

Conflict resolution:

- Both Device A and Device B edit report.pdf offline
- When both sync:
  - Last Write Wins (simple): discard one change – data loss
  - Both versions kept: "report.pdf" + "report (conflict copy).pdf"
  - CRDT (complex): merge changes automatically (like Google Docs)
- Dropbox uses "both versions kept" – simple, no data loss, user resolves

## Architecture



## Quick Reference: Capacity Estimation Cheat Sheet

Common numbers every staff engineer must know:

1 million seconds  $\approx$  11.5 days  
1 billion seconds  $\approx$  31.7 years

Character (UTF-8): 1-4 bytes  
Integer: 4 bytes  
Long: 8 bytes  
UUID: 16 bytes  
Average URL: 100 bytes  
Average tweet: 140 bytes  
Average photo: 2 MB  
Average video: 50 MB (compressed)

Traffic estimation:  
 $1\text{M req/day} = 1\text{M} / 86,400 = \sim 12 \text{ req/sec}$   
 $100\text{M req/day} = \sim 1,200 \text{ req/sec}$   
 $1\text{B req/day} = \sim 12,000 \text{ req/sec}$

Storage estimation:  
 $1\text{M users} \times 1\text{KB each} = 1 \text{ GB}$   
 $1\text{B users} \times 1\text{KB each} = 1 \text{ TB}$   
 $1\text{B photos} \times 2\text{MB each} = 2 \text{ PB}$

Bandwidth:  
 $1,000 \text{ req/sec} \times 100\text{KB} = 100 \text{ MB/sec} = 800 \text{ Mbps}$

Key latency targets:  
LB  $\rightarrow$  service:  $< 1\text{ms}$  (same datacenter)  
Cache hit (Redis):  $< 1\text{ms}$   
DB query (indexed): 1-5ms  
DB query (no index): 100ms-10s  
Cross-region call: 50-150ms  
External API: 50-500ms



---

***Prepared for Apple Inc Staff Engineer Interview | System Design Volume I***

>

Key themes Apple Staff Engineer interviews probe: - **Trade-off reasoning**: Why did you choose X over Y? What are the downsides? - **Scale thinking**: Start with simple design, identify bottlenecks, evolve - **Failure handling**: What happens when each component fails? - **Data consistency**: Where can you tolerate eventual consistency? Where can't you? - **Back-of-envelope**: Can you quickly estimate QPS, storage, bandwidth?